

0.53. Deux lettres, deux carrés

Deux lettres, chacune autour de son propre centre : un A petit et lent, un B grand et rapide.

1. Ce qu'il faut comprendre

Deux lettres, deux centres. Jusqu'ici, tous les carrés tournaient autour de (10, 8). Ici, le A tourne autour de (4, 4), le B autour de (9, 9).

Ce qui est nouveau ici : une **deuxième lettre**, `ALPHABET[1]`, le **B**. Chaque appel à `carre` a sa propre tuile et son propre centre : les deux carrés ne se touchent pas.

Ils se font l'un après l'autre, comme au 0.52 : le B commence quand le A est revenu à son centre. À la fin, on voit les deux lettres, chacune à sa place.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <carre>` → [0.33.3. La fonction carre\(\) — tourner en carré](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1380 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
carre()         440 o ← main() – une fonction fournie par le langage
main()         69 o ← démarrage – ta fonction
VBlank          8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage   28 o ← image() – attend l'image suivante
AttendreVBlank  16 o ← démarrage – attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases – attend de pouvoir écrire une case
AdresseCase     10 o ← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() – efface des cases
CopierTuiles    18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage – vide les deux cartes de l'écran
Tuiles          704 o ← CopierTuiles – la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()   × 4   52 o sur place, dans carre() + AdresseCase (10 o)
effacer() × 3   45 o sur place, dans carre() + AdresseCase (10 o), EffacerCases (12 o)
image()   × 4   12 o sur place, dans carre(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
carre()   × 2   62 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <carre>      // une tuile qui tourne en carré
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // ALPHABET[0] est le A, ALPHABET[1] le B : on compte à partir de 0.

    // 1. Le A : autour de (4, 4), taille 1 (3 × 3), sens 1, 250 ms, 1 tour.
    carre(4, 4, ALPHABET[0], 1, 1, 250, 1);

    // 2. Le B, la ligne nouvelle : autour de (9, 9), taille 3 (7 × 7),
    //    l'autre sens, 100 ms, 1 tour. Il commence quand le A a fini.
    //
    //    colonne : 3  4  5  6 ... 9 ... 12
    //    ligne 4 :  .  A  .                le carré du A : colonnes 3 à 5
    //    ligne 9 :          .  B  .        celui du B : colonnes 6 à 12
    //                                           – ils ne se touchent pas
    carre(9, 9, ALPHABET[1], 3, -1, 100, 1);

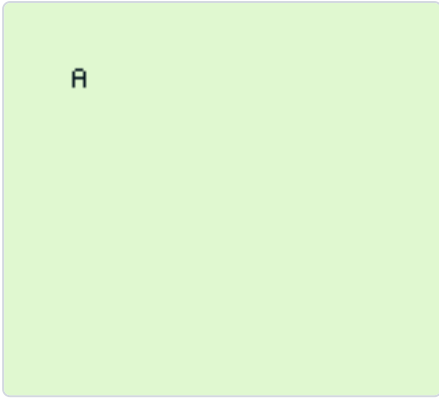
    while (true) {
        image();          // A en (4, 4), B en (9, 9)
    }
}
```

3. Ce qui se passe, étape par étape

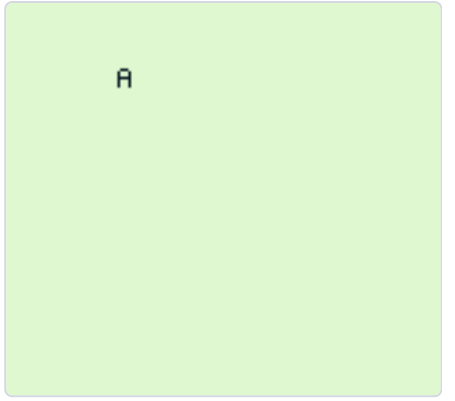
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



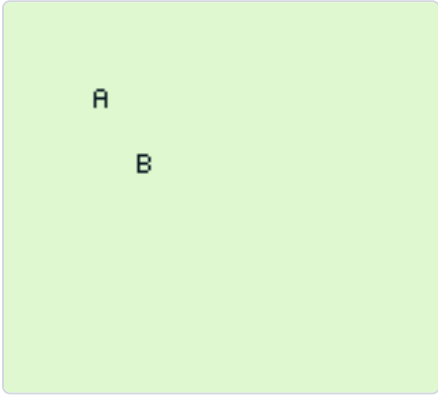
la toute première image — la console vient de commencer — image 1



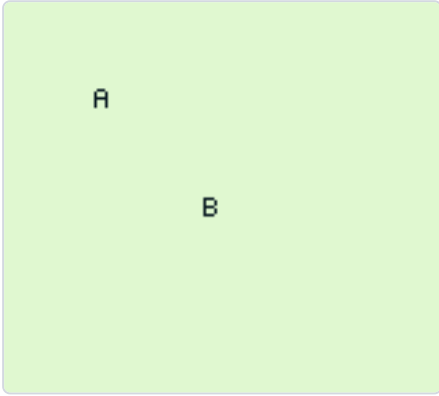
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A qui fait un petit carré autour de (4, 4), puis un B qui fait un grand carré rapide autour de (9, 9). Chacun reste à son centre.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A fait son tour : (3, 3), (5, 5)
- ✓ le B fait le sien, dans l'autre sens : du coin (6, 6), il descend
- ✓ chacun à son centre : A (4, 4), B (9, 9) (4,4 9,9)